

P1950R0

indirect_value: A Free-Store-Allocated Value Type For C++

Published Proposal, 2020-01-13

This version:

wg21.link/P1950r0

Issue Tracking:

[GitHub](https://github.com/jbcoe/indirect_value)

Authors:

[Jonathan B. Coe](#)

[Antony Peacock](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/jbcoe/indirect_value/documentation/p1950.md

Table of Contents

1 Abstract

2 Introduction

2.1 Motivation - a perfect PImpl

2.1.1 Issues with `unique_ptr<impl>` implementation of PImpl Idiom

2.1.1.1 const-propagation

2.1.1.2 copies

2.1.2 Issues with `polymorphic_value<impl>` implementation of PImpl Idiom

2.1.3 Perfect Implementation of the PImpl Idiom

2.2 Prior Art

2.3 Completeness of T*

2.4 Impact on the standard

3 Technical specifications

3.1 X.X Class template `default_copy` [default.copy]

3.2 X.Y Class template `indirect_value` [indirect_value]

3.2.1 X.Y.1 Class template `indirect_value` general [indirect_value.general]

3.2.2 X.Y.2 Class template `indirect_value` synopsis [indirect_value.synopsis]

3.2.3 X.Y.3 Class template `indirect_value` constructors [indirect_value.ctor]

3.2.4 X.Y.4 Class template `indirect_value` destructor [indirect_value.dtor]

3.2.5 X.Y.5 Class template `indirect_value` assignment [indirect_value.assignment]

3.2.6 X.Y.6 Class template `indirect_value` modifiers [indirect_value.modifiers]

3.2.7 X.Y.7 Class template `indirect_value` observers [indirect_value.observers]

3.2.8 X.Z.8 Class template `indirect_value` creation [indirect_value.creation]

4 Acknowledgements

5 References

§ 1. Abstract

Add a class template, `indirect_value`, to the C++ Standard Library to support free-store-allocated objects with value-like semantics.

§ 2. Introduction

The class template, `indirect_value`, confers value-like semantics on a free-store-allocated object. An `indirect_value` may hold an object of a class `T`, copying the `indirect_value` will copy the object `T`, and `const`ness will propagate from the owning classes to the `indirect_value` type.

§ 2.1. Motivation - a perfect PImpl

In C++, when anything in a class definition changes, dependent classes require recompilation. As early as 1992 the Handle/Body idiom was suggested to break this dependency [J. Coplien]. From this pattern, the PImpl idiom was specialised [H. Sutter]. Almost three decades on since these ideas started

circulating implementation of the PImpl idiom requires thought about implementation details around copying and to support this feature requires user intervention.

```
// Header file
class widget {
public:
    widget();
    ~widget();
private:
    class impl;
    std::unique_ptr<impl> pimpl_;
};

// Implementation file
class widget::impl {
    // :::
};

widget::widget() : pimpl_{ std::make_unique<impl>( /*...*/ ) } {}
widget::~~widget() = default;
```

For convenience, the widget class will be referred to as the “visible class” and impl class the “PImpl class”. Note, semantically the PImpl class is the implementation details of the visible class.

§ 2.1.1. Issues with `unique_ptr<impl>` implementation of PImpl Idiom

§ 2.1.1.1. *const-propagation*

Using `std::unique_ptr` to store the implementation object introduces an issue - within a `const`-qualified member function, an instance of the visible class can mutate data inside the implementation object. This is because `const` qualification applies only to the `unique_ptr` value, and not the pointed-to-object.

The compiler is unable to give thread-compatibility guarantees for `const` objects when `const` is not propagate: `const` does not mean immutable in the face of pointer-like-member data.

The desired semantics of a PImpl-owner are value-like, like those of `std::optional` which has appropriate `const` and non-`const`-qualified overloads for `operator*` and `operator->`.

§ 2.1.1.2. *copies*

The copy-constructor and copy-assignment operator of `std::unique_ptr` are deleted. Users of a class with a `std::unique_ptr` member will be required to implement copy-construction and copy-assignment. Implementation of these methods for a class with PImpl members is non-trivial [S. Meyers].

§ 2.1.2. Issues with `polymorphic_value<impl>` implementation of PImpl Idiom

The class template `polymorphic_value` confers value-like semantics on a free-store-allocated object. This satisfies our requirement for const-propagation and a compiler generated copy constructor and copy assignment operator but incurs an additional cost in supporting polymorphism. To implement deep copies of subclasses, `polymorphic_value` employs type-erasure which has run-time cost due to virtual dispatch and memory indirection. We do not need to support polymorphism for our PImpl object and should not pay for what we do not use.

§ 2.1.3. Perfect Implementation of the PImpl Idiom

Like `polymorphic_value`, `indirect_value` confers value-like semantics on a free-store-allocated object. Unlike `polymorphic_value`, `indirect_value` does not support polymorphism - `indirect_value<T>` is either empty or contains an object of type `T`.

```

// Header file
class widget {
public:
    widget();
    widget(widget&& rhs) noexcept;
    widget(const widget& rhs);
    widget& operator=(widget&& rhs) noexcept;
    widget& operator=(const widget& rhs);
    ~widget();
private:
    class impl;
    std::indirect_value<impl>> pimpl;
};

// Implementation file
class widget::impl {
    // :::
};

// Force instantiation of the rule of zero methods in the translation unit
widget::widget(widget&& rhs) noexcept = default;
widget::widget(const widget& rhs) = default;
widget& widget::operator=(widget&& rhs) noexcept = default;
widget& widget::operator=(const widget& rhs) = default;
widget::~~widget() = default;

```

Now the visible class has compiler-generated copy and move constructors, copy and move assignment operators and destructor. All of these compiler-generated functions will behave correctly.

§ 2.2. Prior Art

There have been previous proposal for deep-copying smart pointers that proposed copy semantics [W. Brown]. `cloned_ptr` was proposed in [J. Coe], however under guidance of LEWG this was renamed to `polymorphic_value`. With this change in name came the addition of const propagation.

This paper is not unique in these ideas. GitHub code search finds 602k lines of code referencing "PImpl" and 99 C++ repositories claiming to provide generic implementations of Pimpl. Additionally other authors have addressed this topic [A. Upadyshev].

§ 2.3. Completeness of T*

Recommended practice for pointer-to-implementation of PImpl classes since C++11 has been to store the pointer-to-implementation in a `std::unique_ptr` [S. Meyers]. Smart pointer types in the Standard Library expect that some of the members can be instantiated with incomplete types [H.Hinnant]. Similarly, this is the case for `indirect_value`, the table outlines the expected behaviour for incomplete pointer types:

Method	Description	Incomplete/Complete
<code>indirect_value()</code>	Default constructor	Incomplete
<code>indirect_value(const indirect_value&)</code>	Copy-constructor	Complete
<code>indirect_value(indirect_value&&)</code>	Move-constructor	Incomplete
<code>~indirect_value()</code>	Destructor	Complete
<code>indirect_value& indirect_value::operator=(const indirect_value&)</code>	Copy-assignment	Complete
<code>indirect_value& indirect_value::operator=(indirect_value&&)</code>	Move-assignment	Complete
<code>T& operator*()</code>	Indirection-operator	Incomplete
<code>const T& operator*() const</code>	Indirection-operator	Incomplete
<code>T* operator->() noexcept</code>	Member-of-pointer-operator	Incomplete
<code>const T* operator->() const noexcept</code>	Member-of-pointer-operator	Incomplete
<code>explicit operator bool() const noexcept</code>	Bool-operator	Incomplete
<code>void swap(indirect_value& p) noexcept</code>	Swap	Incomplete

§ 2.4. Impact on the standard

This proposal is a pure library extension. It requires additions to be made to the standard library header `<memory>`.

§ 3. Technical specifications

§ 3.1. X.X Class template `default_copy` [`default.copy`]

```
namespace std {  
    template <class T>  
    struct default_copy {  
        T* operator()(const T& t) const;  
    };  
} // namespace std
```

The class template `default_copy` serves as the default copier for the class template `indirect_value`. The template parameter `T` of `default_copy` may be an incomplete type.

```
T* operator()(const T& t) const;
```

- Returns: `new T(t);`

§ 3.2. X.Y Class template `indirect_value` [`indirect_value`]

§ 3.2.1. X.Y.1 Class template `indirect_value` general [`indirect_value.general`]

An `indirect_value` is an object that owns another object and manages that other object through a pointer. More precisely, an indirect value is an object `v` that stores a pointer to a second object `p` and will dispose of `p` when `v` is itself destroyed (e.g., when leaving block scope (9.7)). In this context, `v` is said to own `p`.

An `indirect_value` object is empty if it does not own a pointer.

Copying a non-empty `indirect_value` will copy the owned object so that the copied `indirect_value` will have its own unique copy of the owned object.

Copying from an empty `indirect_value` produces another empty `indirect_value`.

Copying and disposal of the owned object can be customised by supplying a copier and deleter.

The template parameter `T` of `indirect_value` must be a non-union class type.

The template parameter `T` of `indirect_value` may be an incomplete type.

[Note: Implementations are encouraged to avoid the use of dynamic memory for ownership of small objects.]

§ 3.2.2. X.Y.2 Class template `indirect_value` synopsis [indirect_value.synopsis]

```
template <class T, class C = std::default_copy<T>, class D = std::default_destruct<T>>
class indirect_value {
public:
    using value_type = T;

    // Constructors
    constexpr indirect_value() noexcept;
    explicit indirect_value(T* p, C c=C{}, D d=D{});

    indirect_value(const indirect_value& p);
    indirect_value(indirect_value&& p) noexcept;

    template <class ...Ts>
    indirect_value(std::in_place_t, Ts&&... ts);

    // Destructor
    ~indirect_value();

    // Assignment
    indirect_value& operator=(const indirect_value& p);
    indirect_value& operator=(indirect_value&& p) noexcept;

    // Modifiers
    void swap(indirect_value<T>& p) noexcept;

    // Observers
    T& operator*();
    T* operator->();
    const T& operator*() const;
    const T* operator->() const;
    explicit operator bool() const noexcept;
};

// indirect_value creation
template <class T, class ...Ts> indirect_value<T>
make_indirect_value(Ts&& ...ts);

// indirect_value specialized algorithms
template<class T>
void swap(indirect_value<T>& p, indirect_value<T>& u) noexcept;
```



```
} // end namespace std
```

§ 3.2.3. X.Y.3 Class template `indirect_value` constructors [indirect_value.ctor]

```
constexpr indirect_value() noexcept;
```

- Remarks: The method shall work with incomplete pointer type for `T`.
- Effects: Constructs an empty `indirect_value`.
 - Postconditions: `bool(*this) == false`.

```
explicit indirect_value(T* p, C c=C{}, D d=D{});
```

- Effects: Creates an `indirect_value` object that owns the pointer `p`. If `p` is non-null then the copier and deleter of the `indirect_value` constructed are moved from `c` and `d`.
- Requires: `C` and `D` satisfy the requirements of CopyConstructible. If `p` is non-null then the expression `c(*p)` returns an object of type `T*`. The expression `d(p)` is well-formed, has well-defined behaviour, and does not throw exceptions.
- Postconditions: `bool(*this) == bool(p)`.
- Remarks: A custom copier and deleter are said to be ‘present’ in a `indirect_value` initialised with this constructor.

```
indirect_value(const indirect_value& p);
```

- Constraints: `is_copy_constructible_v<T>`
- Effects: Creates a `indirect_value` object that owns a copy of the object managed by `p`. The copy is created by the copier in `p`. If `p` has a custom copier and deleter then the custom copier and deleter of the `indirect_value` constructed are copied from those in `p`.
- Throws: Any exception thrown by the copier or `bad_alloc` if required storage cannot be obtained.
- Postconditions: `bool(*this) == bool(p)`.

```
indirect_value(indirect_value&& p) noexcept;
```

- Effects: Move-constructs an `indirect_value` instance from `p`. If `p` has a custom copier and deleter then the copier and deleter of the `indirect_value` constructed are the same as those in

`p`.

- Postconditions: `*this` contains the old value of `p`. `p` is empty.
- Remarks: The method shall work with incomplete pointer type for `T`.

```
indirect_value(std::in_place_t, Ts&& ...ts);
```

- Effects: Constructs an `indirect_value` which owns an object of type `T` direct-non-list-initialized with `std::forward<Ts>(ts)...`
- Throws: Any exception thrown by the selected constructor of `T` or `bad_alloc` if required storage cannot be obtained.

§ 3.2.4. X.Y.4 Class template `indirect_value` destructor [`indirect_value.dtor`]

```
~indirect_value();
```

- Effects: If `get() == nullptr` there are no effects. If a custom deleter `d` is present then `d(p)` is called and the copier and deleter are destroyed. Otherwise the destructor of the managed object is called.

§ 3.2.5. X.Y.5 Class template `indirect_value` assignment [`indirect_value.assignment`]

```
indirect_value& operator=(const indirect_value& p);
```

- Constraints: `is_copy_assignable<T>`
- Effects: `*this` owns a copy of the resource managed by `p`. If `p` has a custom copier and deleter then the copy is created by the copier in `p`, and the copier and deleter of `*this` are copied from those in `p`. Otherwise, the resource managed by `*this` is initialised by the copy constructor of the resource managed by `p`.
- Throws: Any exception thrown by the copier or `bad_alloc` if required storage cannot be obtained.
- Returns: `*this`.
- Postconditions: `bool(*this) == bool(p)`.

```
indirect_value& operator=(indirect_value&& p) noexcept;
```

- Effects: Ownership of the resource managed by `p` is transferred to this. If `p` has a custom copier and deleter then the copier and deleter of `*this` is the same as those in `p`.

- Returns: `*this`.
- Postconditions: `*this` contains the old value of `p`. `p` is empty.

§ 3.2.6. X.Y.6 Class template `indirect_value` modifiers [`indirect_value.modifiers`]

```
void swap(indirect_value& p) noexcept;
```

- Effects: Exchanges the contents of `p` and `*this`.
- Remarks: The method shall work with incomplete pointer type for `T`.

§ 3.2.7. X.Y.7 Class template `indirect_value` observers [`indirect_value.observers`]

```
T& operator*();  
const T& operator*() const;
```

- Requires: `bool(*this)`.
- Returns: A reference to the owned object.

```
T* operator->() noexcept;  
const T* operator->() const noexcept;
```

- Requires: `bool(*this)`.
- Returns: A pointer to the owned object.
- Remarks: The method shall work with incomplete pointer type for `T`.

```
explicit operator bool() const noexcept;
```

- Returns: false if the `indirect_value` is empty, otherwise true.
- Remarks: The method shall work with incomplete pointer type for `T`.

§ 3.2.8. X.Z.8 Class template `indirect_value` creation [`indirect_value.creation`]

```
template <class T, class U=T, class ...Ts> indirect_value<T>  
    make_indirect_value(Ts&& ...ts);
```

- Constraints: `is_constructible_v<U, Ts...>` is true.
- Expects: `U` meets the `Cpp17CopyConstructible` requirements.
- Returns: A `indirect_value<T>` owning an object of type direct-non-list-initialized with `std::forward<Ts>(ts)...`.

§ 4. Acknowledgements

The authors would like to thank Thomas Russell, and Andrew Bennieston for useful discussions on the topic and the BSI panel for on-going support.

§ 5. References

[J. Coe] [p0201r3: A polymorphic value-type for C++](#)

[J. Coplien] Advanced C++ Programming Styles and Idioms (Addison-Wesley), James O. Coplien, 1992

[A. Upadyshev] [PIMPL, Rule of Zero and Scott Meyers, Andrey Upadyshev, 2015](#)

[H. Hinnant] [“Incomplete types and shared_ptr / unique_ptr”, Howard Hinnant, 2011](#)

[H. Sutter] ["Pimpls - Beauty Marks You Can Depend On", Herb Sutter, 1998](#)

[Impl] [Reference implementation: indirect_value, J.B.Coe](#)

[S. Meyers] Effective Modern C++, Item 22: When using the Pimpl Idiom, define special member functions in the implementation file, Scott Meyers, 2014

[W. Brown] [n3339: A Preliminary Proposal for a Deep-Copying Smart Pointer, Walter E. Brown, 2012](#)